

Project 4 – Machine Learning Verification

Benjamin Deutsch (12215881)

TU Wien

Collision Avoidance

Project Task 4.1

Property 1 on 1_9. Marabou reports unsat on network 1_9 given ACASXU_property1.txt. This means, that there are no inputs $x_1, \dots, x_4$ within the specified bounds such that the network 1_9 outputs $y_0 \dots, y_4$, such that $y_2 \leq y_i$ for all $i \in \{0, 1, 3, 4, 5\}$. Therefore, in this large vertical separation scenario, the network 1_9 will never advise to turn right strongly.

Property 1 on 2_1. Marabou reports sat on network 2_1 given ACASXU_property2.txt, since it found inputs within the given bounds that result in $y_0 \geq y_i$ for all $i \in \{1, 2, 3, 4, 5\}$.

For the physical aircraft, this means that there could be an extremely distant intruder at significantly slower speed than the ownship, where network 2_1 judges that not evading is the worst option. If the aircraft then followed the network's advice, it would make an unnecessary turn.

This outcome is important for DNNs in aviation, as it reveals that the network 2_1 does not behave as expected in the specified case, which could lead to unsafe situations. The task description notes that previous approaches were validated with 1.5 million simulated tests. Marabou found a specific point in the 5-dimensional input space that violates the expected behaviour. Testing would almost certainly never sample this specific point, highlighting the importance of formal verification of neural networks in safety-critical applications.

MNIST Robustness: Network Size and Scalability

Project Task 5.1

The Epsilon Spectrum. I got the following results when running Marabou on network mnist10x10 with 3600 seconds timeout:

- $\epsilon = 0.005$: unsat (0.064s)
- $\epsilon = 0.05$: timeout
- $\epsilon = 0.1$: sat (0.597s)

The unsat result for $\epsilon = 0.005$ means that the network is robust for this ϵ -range, while the sat result for $\epsilon = 0.1$ means that the network is not robust for this ϵ -range, since Marabou found a counterexample. The results therefore show that the robustness boundary of the network lies strictly between $\epsilon = 0.005$ and $\epsilon = 0.1$. The timeout for $\epsilon = 0.05$ does not allow us to reduce the size of the ϵ -boundary-interval.

The Architecture Comparison. I got the following results when running Marabou on network mnist6x256 with 3600 seconds timeout:

- $\epsilon = 0.005$: unsat (1.344s)
- $\epsilon = 0.05$: sat (91.571s)
- $\epsilon = 0.1$: sat (69.147s)

By the same reasoning as in the previous paragraph, the results show that the robustness boundary of the larger network lies strictly between $\epsilon = 0.005$ and $\epsilon = 0.05$.

Interpretation. The larger network requires significantly more time to verify. For example, at $\epsilon = 0.005$, the smaller network is verified in 0.064s, while the larger one takes 1.344s. More critically, at $\epsilon = 0.1$, the smaller network is verified in 0.597s, while the larger one takes 69.147s. However, counterintuitively, the Marabou was able to verify the larger network at $\epsilon = 0.05$, while it timed out for the smaller one. One speculation for this could be that the robustness boundary of the smaller network is near $\epsilon = 0.05$. Usually, in optimization problems, the closer we are to the boundary, or the optimal solution, the harder verification/solving becomes. Therefore, this could be an explanation for the timeout.

The results show that the robustness boundary interval of the larger network is included in the interval of the smaller network. Therefore, the larger network could be less robust than the smaller one, which is counterintuitive to me. One possible explanation for this could be that the larger network fits the training data more precisely, potentially at the cost of robustness.

In a safety-critical context both accuracy and formal verifiability are essential. A network which is highly accurate on benchmarks but cannot be verified provides no safety guarantees. My approach would be to first set a minimum robustness threshold, driven by the safety requirements of the task. Then, I would select the network that provides the best accuracy on the benchmarks that can be verified for the given threshold.

Final Synthesis

Project Task 6.1

When verifying a general safety rule, like in the ACAS Xu examples, the input space (x constraints) is fixed to an abstract scenario (e.g. intruder at a certain distance, speed, etc.). The output space (y constraints) is then defined by the specific safety rule we want to define, as can be seen from the ACAS Xu examples (e.g. to verify that the network outputs that a certain option is not the worst, we constrain it to be the worst and check satisfiability).

On the other hand, when verifying robustness, like in the MNIST examples, the input space is the ϵ -range around a specific input. For example, if the specific input has $x_0 = 0.1$, then the input space for $\epsilon = 0.05$ would be $x_0 \in [0.05, 0.15]$. Hence, the input constraints are then $0.05 \leq x_0 \leq 0.15$. The output space encodes that some other class scores higher than the correct class (wrong classification). In the MNIST example, image2 represents a 4. Therefore, there are 9 different verification queries for each ϵ , checking whether the 4 is classified as a 0,1,2,3,5,6,7,8 or 9. If all these queries are unsat, then the network can robustly classify the image as a 4, given the ϵ .